PS2 Controller USB Receiver Reception and Processing Source Code Analysis

1. Preface

In the previous lesson, we learned about the analysis of the principles behind the reception and processing of PS2 controller USB receivers. In this lesson, we will continue from where we left off and conduct a source code analysis based on the principles covered in the previous session.

2. Program Storage Path

The source code implemented in this section is located in the 'RRC PS2 Controller USB Receiver Reception and Processing Routine/ rosrobotcontrollerm4_motor_ps2' of Hiwonder/System's 'gampad_handle.c' and 'usbh_hid_gamepad.h'. To open the project files, use Keil5 (for specific installation instructions, please refer to the previous content).

3. Define the Relevant Structures for USB HID Gamepad

3.1 HID_GAMEPAsD_Info_TypeDef Structure

```
typedef struct {
    uint16_t buttons;
  uint8_t hat;
    int8_t lx;
    int8_t ly;
    int8_t rx;
    int8_t ry;
} HID_GAMEPAD_Info_TypeDef;
```

Defined a set of buttons, a switch, as well as the X and Y coordinates of the left and right joysticks.

3.2 SBH_HID_GamepadInit and USBH_HID_GetGamepadInfo Function

```
USBH_StatusTypeDef USBH_HID_GamepadInit(USBH_HandleTypeDef *phost);
HID_GAMEPAD_Info_TypeDef *USBH_HID_GetGamepadInfo(USBH_HandleTypeDef *phost);
```

USBH_HID_GamepadInit is used to initialize data related to the gamepad.

USBH_HID_GetGamepadInfo is used to retrieve information about the gamepad.

3.3 Defined Masks Representing Each Button on the Gamepad

```
#define GAMEPAD_BUTTON_MASK_L2          0x0001u
#define GAMEPAD_BUTTON_MASK_R2          0x0002u
#define GAMEPAD_BUTTON_MASK_SELECT      0x0004u
#define GAMEPAD_BUTTON_MASK_START       0x0008u
#define GAMEPAD_BUTTON_MASK_L3          0x0020u
#define GAMEPAD_BUTTON_MASK_R3          0x0040u
#define GAMEPAD_BUTTON_MASK_CROSS       0x0100u
#define GAMEPAD_BUTTON_MASK_CIRCLE      0x0200u
#define GAMEPAD_BUTTON_MASK_SQUARE      0x0800u
#define GAMEPAD_BUTTON_MASK_TRIANGLE    0x1000u
#define GAMEPAD_BUTTON_MASK_L1          0x4000u
#define GAMEPAD_BUTTON_MASK_R1          0x8000u


#define GAMEPAD_GET_BUTTON(gi, km)  (((gi)->buttons & km) ? true : false)
```

Defined masks representing various buttons on the gamepad, such as L2, R2, SELECT, START, and others.

GAMEPAD_GET_BUTTON is a macro used to check if a given button in the gamepad information is pressed.

4. Program Analysis

4.1 Import Header File

```
#include "usb_host.h"
#include "usbh_core.h"
#include "usbh_hid.h"
#include "usbh_hid_gamepad.h"
#include "cmsis_os2.h"

#define ATC_THRESHOLD 60
```

USB host (usb_host.h and usbh_core.h).

Handling functions for HID devices (usbh_hid.h and usbh_hid_gamepad.h).

As well as the message queue of the CMSIS-RTOS operating system (cmsis_os2.h).

ATC_THRESHOLD defines the threshold for analog joysticks, used to determine if joystick movement exceeds the set threshold.

4.2 A_T_C Function

```
static char A_T_C(int8_t analog_x, int8_t analog_y);
```

This is used to handle inputs from analog joysticks, returning a character representing the direction based on the joystick's x and y coordinates.

It checks if the x coordinate exceeds the set threshold, then checks if the y coordinate exceeds the set threshold, and finally returns a character representing the direction based on the results of these two coordinates.

4.3 USBH_HID_EventCallback Function

```
void USBH_HID_EventCallback(USBH_HandleTypeDef *phost)
{
    extern osMessageQueueId_t moving_ctrl_queueHandle;
    static HID_GAMEPAD_Info_TypeDef last_info;
    static char last_direction_msg = 'I';
    static char last_button = 'R';
```

USBH_HID_EventCallback is a callback function of the USB host library.

This function is called when the status of a USB device changes. Within the function, different processing is performed based on the type of device.

If the device type is a gamepad (0xFF), it retrieves information from the gamepad and processes the status of joysticks and buttons. The processing results are then sent to the message queue.

4.4 USBH_HID_GetDeviceType(phost)

```
switch(USBH_HID_GetDeviceType(phost)) {
    case 0xFF: {/* 手柄数据 */
        HID_GAMEPAD_Info_TypeDef *info = USBH_HID_GetGamepadInfo(phost);
        if(info == NULL) {
            break;
        }
```

This program checks the type of device.

The function retrieves the current connected device type using USBH_HID_GetDeviceType(phost).

If a gamepad's information is obtained using USBH_HID_GetGamepadInfo(phost), it proceeds to the next step of processing if the device type is 0xFF (gamepad); otherwise, it exits.

4.5 Check info->hat Status

```
char direction_msg = ' ';
if(info->hat & 0x08) {
    direction_msg = (((info->hat & 0x07) + 1) & 0x07) + 0x41;
    osMessageQueuePut(moving_ctrl_queueHandle, &direction_msg, 0, 0);
} else {
    if(last_info.hat != info->hat) { /* 只在松开时发送一次松开信号 */
        direction_msg = 'I';
        last_direction_msg = 'I';
        osMessageQueuePut(moving_ctrl_queueHandle, &direction_msg, 0, 0);
    } else {
        direction_msg = A_T_C(info->lx, info->ly);
        if(direction_msg != 'I') {
            osMessageQueuePut(moving_ctrl_queueHandle, &direction_msg, 0, 0);
        } else {
            if(last_direction_msg != 'I') { /* 只在松开时发送一次松开信号 */
                osMessageQueuePut(moving_ctrl_queueHandle, &direction_msg, 0, 0);
            }
        }
        last_direction_msg = direction_msg;
    }
}
```

This segment of the program is used to handle the gamepad's directional keys:

By checking the status of info->hat, it determines whether the gamepad's directional keys are pressed. If pressed, it converts the status of the directional keys into a character and sends this character to the message queue.

4.6 A_T_C(info->lx, info->ly) Function

```
char button = A_T_C(info->rx, info->ry);
```

The above image shows a portion of the program for the gamepad's joysticks.

The function A_T_C(info->lx, info->ly) is called to convert the x and y coordinates of the joystick into a character representing the direction. If the joystick is not moved, the returned character will be 'I', and in this case, there is no need to send a message to the message queue. If the joystick is moved, the character representing the direction is sent to the message queue (as shown below).

```
static char A_T_C(int8_t analog_x, int8_t analog_y)
{
    char result = ' ';
    if(analog_x < -ATC_THRESHOLD) {
        if(analog_y < -ATC_THRESHOLD) {
            result = 'D';
        } else if(analog_y > ATC_THRESHOLD) {
            result = 'B';
        } else {
            result = 'C';
        }
    } else if(analog_x > ATC_THRESHOLD) {
        if(analog_y < -ATC_THRESHOLD) {
            result = 'F';
        } else if(analog_y > ATC_THRESHOLD) {
            result = 'H';
        } else {
            result = 'G';
        }
    } else {
        if(analog_y < -ATC_THRESHOLD) {
            result = 'E';
        } else if(analog_y > ATC_THRESHOLD) {
            result = 'A';
        } else {
            result = 'I';
        }
    }
    return result;
}
```

4.7 last_button Handles Gamepad Buttons

```
    if((last_button != 'I' && button == 'I') || button != 'I') {
      char msg = button - 'A' + 'J';
      osMessageQueuePut(moving_ctrl_queueHandle, &msg, 0, 0);
    }
        last_button = button;
        if(GAMEPAD_GET_BUTTON(info, GAMEPAD_BUTTON_MASK_CIRCLE)) {
            osMessageQueuePut(moving_ctrl_queueHandle, "p", 0, 0);
        }
        if(GAMEPAD_GET_BUTTON(info, GAMEPAD_BUTTON_MASK_SQUARE)) {
            osMessageQueuePut(moving_ctrl_queueHandle, "l", 0, 0);
        }
        /** 按下时触发 */
        if(!GAMEPAD_GET_BUTTON(&last_info, GAMEPAD_BUTTON_MASK_TRIANGLE) && GAMEPAD_GET_BUTTON(info, GAMEPAD_BUTTON_MASK_TRIANGLE)) {
            osMessageQueuePut(moving_ctrl_queueHandle, "j", 0, 0);
        }

        if(!GAMEPAD_GET_BUTTON(&last_info, GAMEPAD_BUTTON_MASK_CROSS) && GAMEPAD_GET_BUTTON(info, GAMEPAD_BUTTON_MASK_CROSS)) {
            osMessageQueuePut(moving_ctrl_queueHandle, "n", 0, 0);
        }
        if(!GAMEPAD_GET_BUTTON(&last_info, GAMEPAD_BUTTON_MASK_SELECT) && GAMEPAD_GET_BUTTON(info, GAMEPAD_BUTTON_MASK_SELECT)) {
            osMessageQueuePut(moving_ctrl_queueHandle, "T", 0, 0);
        }
        if(!GAMEPAD_GET_BUTTON(&last_info, GAMEPAD_BUTTON_MASK_START) && GAMEPAD_GET_BUTTON(info, GAMEPAD_BUTTON_MASK_START)) {
            osMessageQueuePut(moving_ctrl_queueHandle, "S", 0, 0);
        }
        if(!GAMEPAD_GET_BUTTON(&last_info, GAMEPAD_BUTTON_MASK_L1) && GAMEPAD_GET_BUTTON(info, GAMEPAD_BUTTON_MASK_L1)) {
            osMessageQueuePut(moving_ctrl_queueHandle, "a", 0, 0);
        }
        if(!GAMEPAD_GET_BUTTON(&last_info, GAMEPAD_BUTTON_MASK_L2) && GAMEPAD_GET_BUTTON(info, GAMEPAD_BUTTON_MASK_L2)) {
            osMessageQueuePut(moving_ctrl_queueHandle, "b", 0, 0);
        }
        if(!GAMEPAD_GET_BUTTON(&last_info, GAMEPAD_BUTTON_MASK_L3) && GAMEPAD_GET_BUTTON(info, GAMEPAD_BUTTON_MASK_L3)) {
            osMessageQueuePut(moving_ctrl_queueHandle, "c", 0, 0);
        }
        if(!GAMEPAD_GET_BUTTON(&last_info, GAMEPAD_BUTTON_MASK_R1) && GAMEPAD_GET_BUTTON(info, GAMEPAD_BUTTON_MASK_R1)) {
            osMessageQueuePut(moving_ctrl_queueHandle, "d", 0, 0);
        }
        if(!GAMEPAD_GET_BUTTON(&last_info, GAMEPAD_BUTTON_MASK_R2) && GAMEPAD_GET_BUTTON(info, GAMEPAD_BUTTON_MASK_R2)) {
            osMessageQueuePut(moving_ctrl_queueHandle, "e", 0, 0);
        }
        if(!GAMEPAD_GET_BUTTON(&last_info, GAMEPAD_BUTTON_MASK_R3) && GAMEPAD_GET_BUTTON(info, GAMEPAD_BUTTON_MASK_R3)) {
            osMessageQueuePut(moving_ctrl_queueHandle, "f", 0, 0);
        }

        memcpy(&last_info, info, sizeof(HID_GAMEPAD_Info_TypeDef));
        break;
    }
    default:
        break;
}
```

The above image shows a portion of the program for the gamepad buttons.

For each button on the gamepad, it checks whether it is pressed. If a button is pressed, it sends the character representing that button to the message queue. Then, it executes the function associated with that command.

Finally, the function copies the current gamepad information to last_info for use in the next call.